

AI Dataset Explorer (DataPulse AI)

System Architecture & Product Requirement Document (PRD v1.0)

Author: AI Engineering & Product Team **Architecture:** Retrieval-Augmented Generation (RAG) **Target Release:** Q4 2026 (MVP Stage)

Tech Stack: React + FastAPI + Supabase **Vector Engine:** pgvector + Hybrid BM25 **LLM Provider:** Gemini Pro API / OpenAI

DOCUMENT STRUCTURE & TABLE OF CONTENTS

1. Executive Summary & Core Vision

2. User Personas & Use Case Journeys

3. System Architecture & RAG Engine Specifications

4. Production Database Schema (PostgreSQL + pgvector)

5. API Endpoint Specifications (FastAPI)

6. UI/UX Specifications & Wireframe Layouts

7. Non-Functional Requirements & RAG Evaluation

8. 12-Week Development Implementation Roadmap

1. Executive Summary & Product Vision

The **AI Dataset Explorer (DataPulse AI)** is a startup-grade intelligence platform engineered to eliminate dataset discovery friction for Machine Learning students, researchers, and data scientists. Rather than requiring users to manually navigate scattered repositories (Kaggle, Hugging Face, UCI ML Repository, Papers With Code) with basic keyword search, DataPulse AI employs a state-of-the-art **Retrieval-Augmented Generation (RAG) pipeline** combined with hybrid vector search.

Users describe their research objective or machine learning project in plain, natural English (e.g., *"I need an eye disease dataset suitable for detecting diabetic retinopathy with CNNs on low GPU hardware"*). The platform retrieves relevant candidates from an aggregated vector database, evaluates data quality and difficulty, links foundational research papers, and outputs an end-to-end ML project roadmap complete with baseline code boilerplates.

Target Value Proposition & Strategic Goals

- **85% Reduction in Discovery Time:** Replaces hours of manual cross-platform browsing with instantaneous, semantically aligned recommendations.
- **Contextual Matchmaking:** Evaluates domain constraints, label noise, sample volume, and hardware requirements (GPU VRAM vs model size).
- **Architectural Blueprint Generation:** Automatically produces directory structures, PyTorch/TensorFlow preprocessing scripts, evaluation metric suites, and model suggestions.

1.1 Competitive Intelligence Matrix

Feature / Capability	Kaggle	Hugging Face	Papers With Code	DataPulse AI (Our Platform)
Search Paradigm	Keyword / Tags	Filter / Modality	Task / Benchmark	Natural Language Intent (RAG)
Hardware Suitability Analysis	No	Partial	No	Automated (GPU VRAM Estimator)
Cross-Repository Indexing	Siloed (Kaggle only)	Siloed (HF only)	Academic focus	Aggregated Multi-Platform Metadata

Feature / Capability	Kaggle	Hugging Face	Papers With Code	DataPulse AI (Our Platform)
Automated ML Roadmap & Code	Community Notebooks	Space Templates	Repo Links	Tailored Python Code & Architecture
Data Quality Metrics	Usability Rating	Card Metadata	Paper Leaderboards	Automated Profiling & EDA Visuals

2. Target User Personas & Use Case Journeys

Persona A: Undergraduate CS Student

Goal: Complete a semester computer vision or NLP capstone project in 6 weeks without getting stuck on data cleaning.

Pain Point: Finds datasets with missing labels, huge file sizes (>50GB) exceeding Google Colab limits, or complex custom formats.

Platform Solution: Beginner-friendly filtering, automatic Colab-ready notebook generation, sample previews, and < 2GB dataset highlighting.

Persona B: AI Researcher / Postdoc

Goal: Locate standardized benchmark datasets in domain-specific tasks (e.g., medical imaging, cyber intrusion) with verified citations.

Pain Point: Licensing ambiguities, unknown class imbalances, lack of baseline benchmarks in published literature.

Platform Solution: Exact license badges, direct citation links, BibTeX exporters, and state-of-the-art (SOTA) accuracy leaderboards.

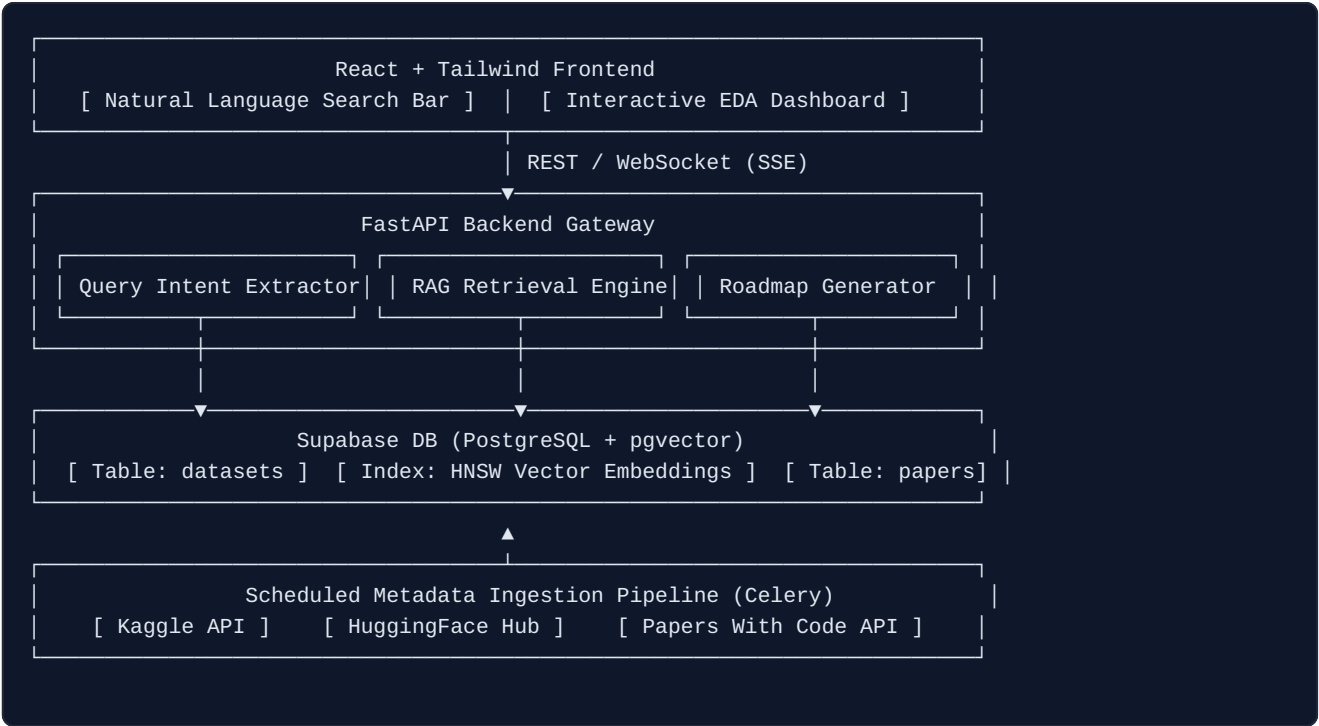
2.1 End-to-End User Interaction Flow

- 1 Natural Language Input:** User enters prompt: *"I want to build an automated chest X-ray classification pipeline to detect pneumonia on a single RTX 3060."*
- 2 Query Intent Parsing:** FastAPI backend parses key constraints: Task: Medical Imaging / Multi-label Classification , Domain: Chest X-Ray , Constraint: RTX 3060 (12GB VRAM) .
- 3 Hybrid Vector Search (pgvector + BM25):** System queries PostgreSQL vector DB for metadata candidates matching embeddings and sparse filters.
- 4 LLM Context Synthesis:** Gemini Pro synthesizes candidate metadata into ranking, rationale, and recommended CNN architectures (e.g., EfficientNet-B0 vs ResNet50).
- 5 Roadmap & Code Generation:** Platform renders dataset intelligence page, EDA visual summary, PyTorch dataloader, and project folder template.

3. System Architecture & RAG Pipeline Specifications

The system follows a modular microservice-inspired architecture built on FastAPI, Supabase (PostgreSQL + pgvector), and modern frontend tooling (React + Tailwind).

High-Level System Topology



3.1 RAG Retrieval Engine Workflow

To ensure accuracy and zero hallucination of dataset links or metrics, the system implements a strict **Hybrid Dense-Sparse Retrieval Framework with Reranking**:

- **Embedding Model:** `text-embedding-004` (768 dimensions) for semantic text vectorization of dataset summaries and task descriptions.
- **Dense Vector Search:** PostgreSQL `pgvector` using HNSW (Hierarchical Navigable Small World) index with cosine distance metric for sub-50ms retrieval speed.
- **Sparse Keyword Search:** PostgreSQL Full-Text Search (`tsvector` / BM25 equivalent) to capture exact dataset titles (e.g., "MIMIC-CXR", "HAM10000", "CIFAR-100").
- **Reciprocal Rank Fusion (RRF):** Combines dense and sparse search scores using the formula:

$$RRF_Score(d) = 1 / (60 + Rank_dense(d)) + 1 / (60 + Rank_sparse(d))$$

- **Cross-Encoder Reranking:** Top 25 retrieved candidates are passed to a lightweight Cross-Encoder model to score deep contextual alignment with user hardware and task constraints.

3.2 LLM Recommendation System Prompting Template

```
SYSTEM PROMPT: DataPulse AI Dataset & Architecture Assistant
-----
You are an expert Machine Learning System Architect. You are given a user request
and a JSON list of retrieved, validated dataset candidates from our PostgreSQL database.

USER QUERY: "{user_query}"
RETRIEVED DATASET CONTEXT:
{retrieved_datasets_json}

RULES:
1. ONLY recommend datasets from the provided JSON context. Never invent dataset names.
2. For each recommendation, explicitly state:
   - Primary suitability rationale.
   - Recommended ML/DL model baseline (e.g., ResNet-34, LightGBM, RoBERTa).
   - Expected accuracy benchmark in academic literature.
   - Hardware requirement (Estimated GPU VRAM, RAM).
   - Preprocessing steps required (e.g., resizing, normalization, handling class imbalance).
3. Generate a structured JSON response conforming strictly to the Roadmap Output Schema.
-----
```

4. Production Database Schema (PostgreSQL + pgvector)

The schema is normalized to decouple dataset metadata, semantic vector embeddings, research papers, and cached roadmap plans.

Table Name	Primary Key	Key Columns & Indexes	Description
datasets	id (uuid)	slug, title, source_platform, size_mb, license, difficulty_level	Core record storing platform metadata, download URLs, and tags.
dataset_features	id (uuid)	dataset_id (FK), num_samples, num_classes, missing_values_pct, modality	Data quality attributes, modality (Images/CSV/Audio/Text), and structure stats.
dataset_embeddings	id (uuid)	dataset_id (FK), embedding (vector(768)), metadata_chunk	Vector embeddings indexed with pgvector HNSW for semantic RAG search.
research_papers	id (uuid)	title, arxiv_id, paper_url, citations_count, year	Linked academic research papers establishing SOTA benchmarks.
project_roadmaps	id (uuid)	user_id (FK), query_text, dataset_id (FK), architecture_json	Generated user ML project plans, folder trees, and code snippets.

4.1 SQL DDL Implementation

```
-- Enable Vector Extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Primary Datasets Table
CREATE TABLE datasets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug VARCHAR(255) UNIQUE NOT NULL,
  title VARCHAR(255) NOT NULL,
  source_platform VARCHAR(50) NOT NULL, -- 'kaggle', 'huggingface', 'uci', 'paperswithcode'
  download_url TEXT NOT NULL,
  license VARCHAR(100) DEFAULT 'Unknown',
  size_mb NUMERIC(10,2) NOT NULL,
  modality VARCHAR(50) NOT NULL, -- 'Computer Vision', 'Tabular', 'NLP', 'Audio'
  difficulty_level VARCHAR(20) CHECK (difficulty_level IN ('Beginner', 'Intermediate', 'Advanced')),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Dataset Vector Embedding Storage
CREATE TABLE dataset_embeddings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  dataset_id UUID REFERENCES datasets(id) ON DELETE CASCADE,
  content_chunk TEXT NOT NULL,
  embedding vector(768) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Create HNSW Index for Fast Vector Retrieval
CREATE INDEX idx_dataset_embeddings_hnsw
ON dataset_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

5. API Endpoint Specifications (FastAPI)

5.1 Semantic RAG Search & Recommendation Endpoint

POST /api/v1/search/semantic

Sample Request Body

```
{
  "query": "I want to detect diabetic retinopathy using deep learning on a budget GPU",
  "filters": {
    "max_size_mb": 5000,
    "modality": "Computer Vision",
    "difficulty": ["Beginner", "Intermediate"]
  },
  "top_k": 5
}
```

Sample Response Payload

```
{
  "status": "success",
  "query": "I want to detect diabetic retinopathy using deep learning on a budget GPU",
  "recommendations": [
    {
      "dataset_id": "c7a8109d-1284-4822-b210-918239021a81",
      "title": "Aptos 2019 Blindness Detection",
      "platform": "Kaggle",
      "size_mb": 3600.0,
      "modality": "Computer Vision",
      "match_score": 0.942,
      "ai_rationale": "Perfect match: Contains 3,662 high-resolution retinal images with 5 ordinal severity grades",
      "suggested_models": ["ResNet-34", "EfficientNet-B0"],
      "expected_accuracy_range": "88% - 94% QWK",
      "gpu_requirement": "4GB - 8GB VRAM"
    }
  ]
}
```

5.2 ML Project Roadmap Generator Endpoint

POST /api/v1/roadmap/generate

Generates a full starter repo structure, PyTorch boilerplate code, data augmentation pipeline suggestions, and paper links for a specific selected dataset ID.

6. UI/UX Design System & Layout Architecture

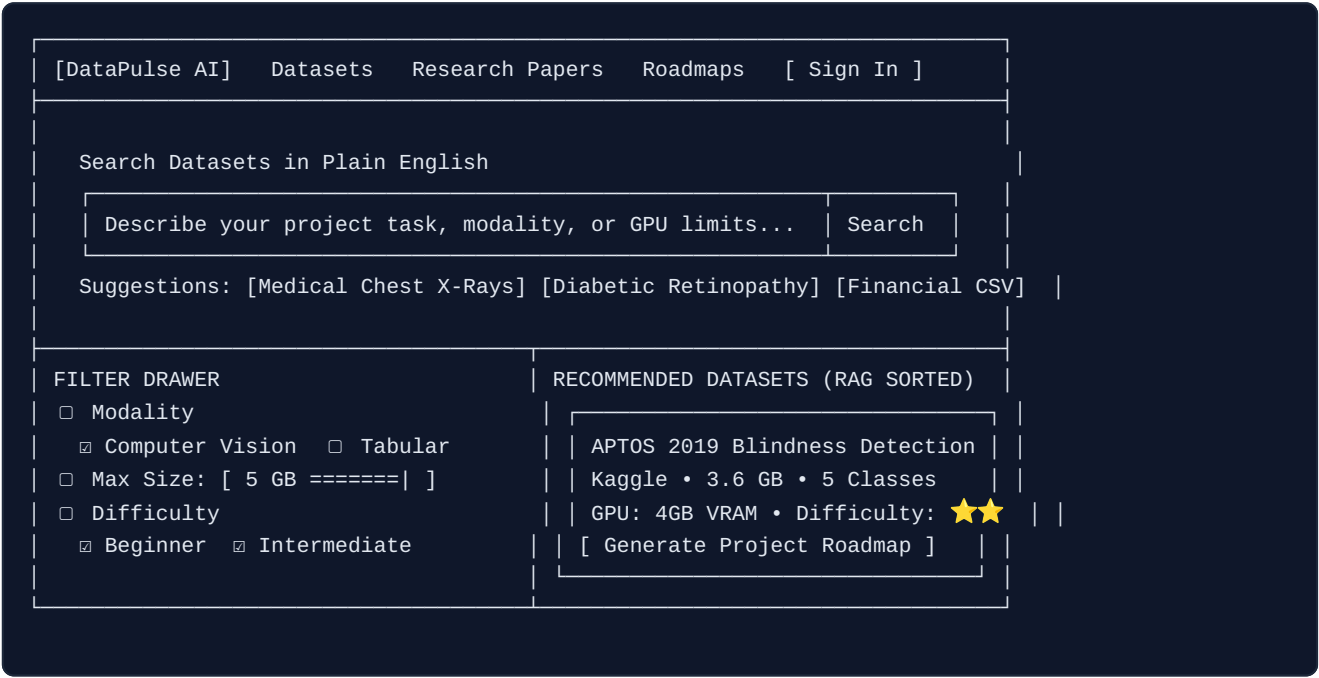
Design System Tokens

- **Primary Color:** Deep Navy (#0F172A)
- **Accent Color:** Indigo Pulse (#4F46E5)
- **Success State:** Emerald (#059669)
- **Warning State:** Amber (#D97706)
- **Background:** Off-white Slate (#F8FAFC)
- **Typography:** Inter / JetBrains Mono (Code)

Core Dashboard Components

- **Prompt Hero Bar:** Multi-line natural language text area with suggestion tags.
- **Dataset Intelligence Card:** Modality badges, size indicator, GPU VRAM bar, download button.
- **Split Preview Panel:** Embedded CSV sample preview / Image gallery grid + Class distribution charts.
- **Roadmap Exporter:** One-click export to GitHub Template or Download `.ipynb` notebook.

6.1 Screen Layout Schematic: Search & Roadmap View



7. Non-Functional Requirements & RAG Evaluation

Category	Target Requirement	Implementation Mechanism
Search Latency	< 200 ms for hybrid search retrieval	Supabase <code>pgvector</code> HNSW index + Redis query caching.
RAG Synthesis Speed	< 2.0 s end-to-end LLM response	Gemini Pro API stream response via Server-Sent Events (SSE).
System Scalability	100,000+ indexed datasets metadata	Celery batch scraper workers + PG partitioning.
Data Privacy & Auth	Role-Based Access Control (RBAC)	Supabase Auth JWT tokens + Row Level Security (RLS).
RAG Guardrails		

Category	Target Requirement	Implementation Mechanism
	Zero hallucination rate on dataset links	Context strictness validation; non-retrieved links are suppressed.

7.1 Continuous Evaluation Framework (RAGAS Metrics)

RAG Automated Evaluation Matrix

- **Faithfulness Metric (> 0.95):** Ensures the LLM's explanations strictly derive from retrieved PostgreSQL metadata.
- **Answer Relevance (> 0.90):** Verifies that dataset recommendations directly fulfill user domain constraints (e.g. VRAM size, task).
- **Context Precision (> 0.88):** Evaluates whether top-ranked vector search results contain relevant dataset candidates.

8. 12-Week Development Roadmap & Implementation Plan

Phase	Timeline	Key Deliverables & Engineering Focus	Milestone Target
Phase 1	Weeks 1–3	Setup Supabase PostgreSQL, schema migration, pgvector extension, metadata scraper scripts for Kaggle API & Hugging Face.	DATA ENGINE READY 10,000 Datasets Indexing
Phase 2	Weeks 4–6	Build FastAPI RAG service, hybrid BM25 + dense search retriever, LLM prompt pipeline, and evaluation suite (Ragas framework).	BACKEND CORE ALPHA Sub-200ms Search Latency
Phase 3	Weeks 7–9	Develop React + Tailwind UI, natural language search bar, dataset cards, interactive EDA chart previews, and code generation drawer.	FULL STACK INTEGRATION Interactive Frontend MVP
Phase 4	Weeks 10–12	User testing with CS students, performance tuning, rate limiting, Vercel/Fly.io deployment, and portfolio presentation prep.	PUBLIC LAUNCH Live Startup MVP Release